Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

BAGYA VENKATESH
192324263RA

CO3-AT3-Debugging and Optimization

View Only

← Back

QUESTIONS

Q1
ATM Withdrawal – Exception Handling Debugging
10 marks

Q2
Hospital Registration – NullPointerException Debugging
10 marks

Q3
E-Commerce Product Search – Array Exception Debugging
10 marks

Q4
Bank Account – Race Condition Debugging
10 marks

Q5
Railway Reservation – Synchronization Debugging
10 marks

5/5 answered

Q4 Bank Account – Race Condition Debugging

10 marks

A bank account contains ₹10,000. Two ATM threads simultaneously attempt to withdraw ₹7,000 and ₹6,000.

The following program contains a race condition.

Task:
1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class BankAccount {

int balance = 10000;

void withdraw(int amount) {

if (balance >= amount) {

System.out.println(
Thread.currentThread().getName()
+ " withdrawing " + amount);
```

```
try {
Thread.sleep(100);
}
catch (InterruptedException e) {
System.out.println("Interrupted");
}

balance = balance - amount;

System.out.println(
Thread.currentThread().getName()
+ " withdrawal successful");
}
else {
System.out.println(
Thread.currentThread().getName()
+ " insufficient balance");
}
}
}

class ATM extends Thread {

BankAccount account;
int amount;

ATM(BankAccount account, int amount) {
this.account = account;
this.amount = amount;
}

public void run() {
account.withdraw(amount);
}
}

class BankApplication {

public static void main(String[] args)
throws InterruptedException {

BankAccount account = new BankAccount();

ATM atm1 = new ATM(account, 7000);
ATM atm2 = new ATM(account, 6000);
```

atm1.setName("ATM-1");
atm2.setName("ATM-2");

atm1.start();
atm2.start();

atm1.join();
atm2.join();

System.out.println(
"Final Balance = " + account.balance);
}
}

Correct the program so that only one withdrawal succeeds.

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 class BankAccount{
2     int balance=10000;
3 
4     synchronized void withdraw(int amount)
5         if(balance>=amount){
6             System.out.println(Thread.curr
7             try{Thread.sleep(100);}catch(I
8             balance-=amount;
9             System.out.println(Thread.curr
10 
11        }else{
12            System.out.println(Thread.curr
13 
14    }
15 }
16 public static void main(String[] args)thro
17    BankAccount account =new BankAccount()
18    ATM atm1=new ATM(account,7000);
19    ATM atm2=new ATM(account,6000);
20    atm1.setName("ATM-1");
21    atm2.setName("ATM-2");
22    atm1.start();
23    atm2.start();
24    atm1.join();
```

```
25      atm2.join();
26      System.out.println("Final Balance = "-
27      }
28  }
29  class ATM extends Thread{
30      BankAccount account;
31      int amount;
32      ATM(BankAccount account,int amount){
33          this.account=account;
34          this.amount=amount;
35
36      }
37      public void run(){
38          account.withdraw(amount);
39      }
40  }
```

Test Input

stdin input (optional)

Output

```
ATM-2 withdrawing6000
ATM-2 withdrawal successful
ATM-1 insuffient balance
Final Balance = 4000
```

Pending manual review

Submitted